Memo: Configuring procedure on Ubuntu 22.04

Initialize environment

1. git clone each repo
2. `sudo apt update`

NodeJS, npm

```
sudo apt install -y nodejs npm
sudo npm install n -g
sudo n 16.16.0
```

Yarn

```
curl -sS https://dl.yarnpkg.com/debian/pubkey.gpg | sudo apt-key add -
ls -la /etc/apt/sources.list.d
echo "deb https://dl.yarnpkg.com/debian/ stable main" | sudo tee /etc/apt/sourc
ls -la /etc/apt/sources.list.d
cat /etc/apt/sources.list.d/yarn.list
sudo apt update
sudo apt install --no-install-recommends yarn
which yarn
yarn --version
```

Install nginx

(Ref: https://www.digitalocean.com/community/tutorials/how-to-install-nginx-on-ubuntu-22-04)

`sudo apt install nginx`

postgreSQL

(Ref: https://phoenixnap.com/kb/how-to-install-postgresql-on-ubuntu)

1. Run:

```
sudo apt install postgresql postgresql-contrib
psql --version
sudo su - postgres
psql postgres
```

2. inside postgres cli:

```
ALTER USER postgres WITH ENCRYPTED PASSWORD 'postgres';
ALTER USER postgres WITH SUPERUSER;
```

asdf

(Ref: https://github.com/asdf-vm/asdf-erlang?tab=readme-ov-file#before-asdf-install)

1. `sudo apt-get -y install zip unzip build-essential autoconf m4 libncurses5-dev libwxgtk3.0-gtk3-dev libwxgtk-webview3.0-gtk3-dev libgl1-mesa-dev libglu1-mesa-dev libpng-dev libssh-dev unixodbc-dev xsltproc fop libxml2-utils libncurses-dev openjdk-11-jdk`
2. `git clone https://github.com/asdf-vm/asdf.git ~/.asdf --branch v0.14.0`
3. Add lines in `~/.bashrc`:

```
# ~/.bashrc
. "$HOME/.asdf/asdf.sh"
. "$HOME/.asdf/completions/asdf.bash"
```

4. `source ~/.bashrc`
5. Run:

```
asdf plugin add erlang
asdf plugin add elixir
```

```
# Erlang 23.3 require openssl 1. but ubuntu 22 come with openssl 3
# solution ref: https://github.com/asdf-vm/asdf-erlang/issues/247#issuecommer

sudo apt-get install libz-dev
cd /usr/local/src/
sudo git clone https://github.com/openssl/openssl.git -b OpenSSL_1_1_1-stable op
cd openssl-1.1.1m
sudo ./config --prefix=/usr/local/ssl --openssldir=/usr/local/ssl shared zlib
sudo make
sudo make test
sudo make install_sw
export KERL_CONFIGURE_OPTIONS="-with-ssl=/usr/local/ssl"
```

```
# and then install erlang
cd ~/chutvrc/chutvrc-reticulum
asdf install
asdf current
```

(output:
elixir 1.14.3-otp-23 /home/username/chutvrc/chutvrc-reticulum/.tool-versions
erlang 23.3.4.18 /home/username/chutvrc/chutvrc-reticulum/.tool-versions
)

python

1. `sudo apt install -y python3-pip`
2. Add lines in `~/.bashrc`:

```
# ~/.bashrc
alias python="python3"
alias pip="pip3"
```

3. `source ~/.bashrc`

Reticulum

1. cd into reticulum folder
2. (20250321): use git branch `fix/switch-sfu-from-frontend`
3. create file: `config/dev.secret.exs`
4. edit `config/dev.secret.exs`

```
use Mix.Config

config :ret, Ret.SoraChannelResolver,
  bearer_token: "", # fill in your sora bearer token
  project_id: "" # fill in your sora project id

config :ret, Ret.Mailer,
  username: "", # fill in your email address
  password: "" # fill in your email account password
```

5. Run:

```
mix deps.get
sudo su - postgres
psql postgres
```

5. inside postgres cli: `ALTER USER postgres WITH PASSWORD 'postgres';`

   Then exit
6. Run:

```
mix ecto.create
mkdir -p storage/dev
```

7. (If you are running chutvrc not only at local) scripts/run-dev.sh:

```
PERMS_KEY=$MOZILLA_RETICULUM_PERMS_PRIVATE_KEY
HUBS_CLIENT_INTERNAL_HOSTNAME="your.PRIVATE.ip.address" HUBS_ADMIN
```

8. Create key for reticulum and dialog
   (Ref: https://github.com/albirrkarim/mozilla-hubs-installation-detailed?tab=readme-ov-file#122-setting-up-secret-key)
   a. Create key pair using Online RSA Key Generator
   b. Paste the public key to Dialog's `certs/perms.pub.pem`
   c. Configure the private as follows:
      i. Add line in `~/.bashrc`:

```
export MOZILLA_RETICULUM_PERMS_PRIVATE_KEY="-----BEGIN RS
```

      ii. (not necessary but) Add line in config/dev.exs:

```
config :ret, Ret.PermsToken, perms_key: "-----BEGIN RSA PRIVATE K
```

9. Fix redirection path:

```
# reticulum/lib/ret_web/controllers/page_controller.ex
# Change this:
  def render_for_path("/", params, conn) do
    if !Enum.empty?(params) || Ret.Account.has_accounts?() do
      conn |> render_index
    else
      conn |> redirect(to: "/admin")
    end
  end

# To this:
  def render_for_path("/", params, conn) do
```

```
    render_index
  end
```

10. config/dev.exs:

```
import Config

# NOTE: this file contains some security keys/certs that are *not* secrets, an

host = "your.domain"
cors_proxy_host = "hubs-proxy.local"
assets_host = "hubs-assets.local"
link_host = "hubs-link.local"
dev_janus_host = "your.domain"
cors_proxy_host = "your.domain"
# To run reticulum on localhost, , uncomment and change the line below to "l
# To run reticulum across a LAN for local testing, uncomment and change the
# host = cors_proxy_host = dev_janus_host = "localhost"

import_config "dev.secret.exs"

# ...
```

10. (If you are running chutvrc not only at local) edit `lib/ret_web/plugs/add_csp.ex`:

```
"connect-src" => [
      "https://your.PRIVATE.ip.address:9090", # add this line
      "wss://your.PRIVATE.ip.address:8989", # add this line
```

Spoke

1. cd into spoke folder
2. `yarn install`
3. Create file `.env`:

```
HUBS_SERVER="your.domain"
RETICULUM_SERVER="your.domain"
THUMBNAIL_SERVER="nearspark-dev.reticulum.io"
NON_CORS_PROXY_DOMAINS="localhost,your.domain,your.domain:4000"
CORS_PROXY_SERVER=""
GITHUB_REPO="spoke"
IS_MOZ="false"
# If running on local
HOST_IP="localhost"
# If running on dev/prod
HOST_IP="your.PRIVATE.ip.address"
```

4. edit `webpack.config.js`:

```
module.exports = env => {
  return {
    // ...
    devServer: {
      // ...
      // allowedHosts: [host, internalHostname], // COMMENT OUT THIS LINE
      disableHostCheck: true, // ADD THIS LINE
```

5. (If you are running chutvrc not only at local) edit/create `scripts/run-dev-reticulum.sh`:

```
#!/usr/bin/env bash
NODE_TLS_REJECT_UNAUTHORIZED=0 ROUTER_BASE_PATH=/spoke BASE_AS
```

Dialog

1. cd to Dialog folder
2. `npm ci`
3. (If you are running chutvrc not only at local) add line to `package.json`:

```
"prod": "MEDIASOUP_LISTEN_IP=0.0.0.0 MEDIASOUP_ANNOUNCED_IP=your.PU
```

Hubs

1. cd to Hubs folder
2. `npm ci`
3. Create `.env`:

```
# See here for the server code: https://github.com/mozilla/reticulum
RETICULUM_SERVER="your.domain"

# CORS proxy.
CORS_PROXY_SERVER="hubs-proxy.com"

# The thumbnailing backend to connect to.
# See here for the server code: https://github.com/MozillaReality/farspark or http
THUMBNAIL_SERVER="nearspark-dev.reticulum.io"

# The root URL under which Hubs expects environment GLTF bundles to be serv
ASSET_BUNDLE_SERVER="https://asset-bundles-prod.reticulum.io"

# Comma-separated list of domains which are known to not need CORS proxying
NON_CORS_PROXY_DOMAINS="localhost,your.PUBLIC.ip.address,your.domain,

# The root URL under which Hubs expects static assets to be served.
BASE_ASSETS_PATH="https://your.domain:8080/" # or "/" if running at local

# If running on local
INTERNAL_HOSTNAME="localhost"
HOST_IP="localhost"

# If running on dev/prod
INTERNAL_HOSTNAME="your.domain"
```

```
HOST_IP="your.domain"

SORA_PUBLIC_SPEAKING_CHANNEL_TOKEN="your.sora.token.for.public.speaki
```

Admin

1. cd to Admin folder (Under Hubs folder)
2. `npm ci`
3. Create `.env`:

```
# To override these variables, create a .env file containing the overrides.

# Services we should configure in this UI.
CONFIGURABLE_SERVICES="janus-gateway,reticulum,hubs,spoke"

# The Ita service, for configuration schemas and updates. (In the future this
# will probably be proxied through Reticulum.)

ITA_SERVER="https://your.domain:3333" # or "https://localhost:3333"

# The Reticulum backend to connect to. Used for storing information about activ
# See here for the server code: https://github.com/mozilla/reticulum
RETICULUM_SERVER="your.domain:4000" # or "https://localhost:4000"

# PostgREST server configured to allow administrative access to the db.
# POSTGREST_SERVER="https://localhost:4000/api/postgrest"
POSTGREST_SERVER="https://your.domain/api/postgrest" # not your.domain:40

# BASE_ASSETS_PATH="https://localhost:8989/"
BASE_ASSETS_PATH="https://your.domain:8989/"

# If running on local
INTERNAL_HOSTNAME="localhost"
HOST_IP="localhost"
```

```
# If running on dev/prod
INTERNAL_HOSTNAME="your.PRIVATE.ip.address" # "your.domain" also works?
HOST_IP="your.PRIVATE.ip.address" # "your.domain" also works?
```

Setup SSL

local

generate and configure ssl certificates following the instruction here: https://github.com/albirrkarim/mozilla-hubs-installation-detailed?tab=readme-ov-file#3-setting-up-https-ssl (Section **3. Setting up HTTPS (SSL)**)

dev/prod

1. generate and configure ssl certificates using following the instruction here: https://github.com/albirrkarim/mozilla-hubs-installation-detailed/blob/main/VPS_FOR_HUBS.md#3-setting-up-https-for-your-domain (Section **3. Setting up HTTPS for Your Domain**)
2. Update config in Reticulum

```
# config/dev.exs

config :ret, RetWeb.Endpoint,
  # ...
  https: [
    # ...
    keyfile: "/etc/letsencrypt/live/your.domain/privkey.pem", # "#{File.cwd!()}/priv/
    certfile: "/etc/letsencrypt/live/your.domain/fullchain.pem", # "#{File.cwd!()}/pr
  ],
  # ...
```

3. Update config in **Hubs**, **Admin**, **Spoke**

```
// webpack.config.js

function createHTTPSConfig() {
  // Add this if-block in the beginning of the createHTTPSConfig function
  if (fs.existsSync("/etc/letsencrypt/live/your.domain/")) {
    const key = fs.readFileSync("/etc/letsencrypt/live/your.domain/privkey.pem");
    const cert = fs.readFileSync("/etc/letsencrypt/live/your.domain/fullchain.pem"

    return { key, cert };
  }
  ...
}
```

4. Confirm Dialog's `package.json` for ssl cert file paths:
   confirm whether the paths of HTTPS_CERT_FULLCHAIN and HTTPS_CERT_PRIVKEY match the values below:

```
// package.json
"prod": "MEDIASOUP_LISTEN_IP=0.0.0.0 MEDIASOUP_ANNOUNCED_IP=your.PU
```

postgREST (inside reticulum)

1. Run:

```
sudo apt install libpq-dev
wget https://github.com/PostgREST/postgrest/releases/download/v9.0.0/post
grest-v9.0.0-linux-static-x64.tar.xz
tar -xf postgrest-v9.0.0-linux-static-x64.tar.xz
```

2. cd to Reticulum folder
2. run Reticulum (temporarily) using `./scripts/run-local.sh` or `./scripts/run-dev.sh`
   a. This will fail if ssl cert files are not configured well, so make sure to configure ssl first

3. Run the following lines inside reticulum iex:
   Stay at the terminal where reticulum is running, press enter key, and then paste the following line, then press enter key

   ```
   jwk = Application.get_env(:ret, Ret.PermsToken)[:perms_key] |> JOSE.JWK.from_pem(); JOSE.JWK.to_file("reticulum-jwk.json", jwk)
   ```

4. Terminate Reticulum: press `ctrl+c` and then `a` key
5. Create/Edit `reticulum.conf` :

```
# reticulum.conf
db-uri = "postgres://postgres:postgres@localhost:5432/ret_dev"
db-schema = "ret0_admin"
db-anon-role = "postgres_anonymous"
jwt-secret = "@/path_from_reticulum_root_folder_to_your_file/reticulum-jwk.json"
jwt-aud = "ret_perms"
role-claim-key = ".postgrest_role"
```

Nginx

Run `sudo vim /etc/nginx/sites-available/default` and replace the content

```
server {
        listen [::]:443 ssl ipv6only=on; # managed by Certbot
        listen 443 ssl; # managed by Certbot
        server_name your.domain; # managed by Certbot

        client_max_body_size 20M;

        ssl_certificate /etc/letsencrypt/live/your.domain/fullchain.pem; # managed b
        ssl_certificate_key /etc/letsencrypt/live/your.domain/privkey.pem; # manage
        include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
        ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot
```

```
        location / {
                # Directly proxy all requests to your backend
                rewrite ^\/(.*)$ /$1 break;
                proxy_pass https://your.domain:4000/;

                # Pass original headers to the backend
                proxy_set_header Host $host;
                proxy_set_header X-Real-IP $remote_addr;
                proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
                proxy_set_header X-Forwarded-Proto $scheme;

                # WebSocket support
                proxy_http_version 1.1;
                proxy_set_header Upgrade $http_upgrade;
                proxy_set_header Connection "upgrade";

                proxy_ssl_server_name on;
                proxy_ssl_verify off;

                #Give larger upstream buffers
                fastcgi_buffers 16 16k;
                fastcgi_buffer_size 32k;
                proxy_buffer_size 128k;
                proxy_buffers 4 256k;
                proxy_busy_buffers_size 256k;
        }
}

server {
        listen 80 ;
        listen [::]:80 ;
        server_name your.domain;

        if ($host = your.domain) {
                return 301 https://$host$request_uri;
        } # managed by Certbot
```

```
    return 404; # managed by Certbot
}
```

▼ (deprecated)

```
server {
        client_max_body_size 20M;
        server_name chutvrc.vr.u-tokyo.ac.jp; # managed by Certbot


        location / {
                # First attempt to serve request as file, then
                # as directory, then fall back to displaying a 404.
                # try_files $uri $uri/ =404;

                #match everything
                rewrite ^\/(.*)$ /$1 break;
                # Proxy passing to port 4000
                proxy_pass https://chutvrc.vr.u-tokyo.ac.jp:4000;


                # The Important Websocket Bits!
                proxy_http_version 1.1;
                proxy_set_header Upgrade $http_upgrade;
                proxy_set_header Connection "upgrade";

                #Give larger upstream buffers
                fastcgi_buffers 16 16k;
                fastcgi_buffer_size 32k;
                proxy_buffer_size 128k;
                proxy_buffers 4 256k;
                proxy_busy_buffers_size 256k;
        }
```

```
    listen [::]:443 ssl ipv6only=on; # managed by Certbot
    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/chutvrc.vr.u-tokyo.ac.jp/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/chutvrc.vr.u-tokyo.ac.jp/privkey.pe
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot

}

server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name chutvrc.vr.u-tokyo.ac.jp;

    #hubs must not serve with http, so redirect it to https
    return 301 https://$host$request_uri;
}
```

Execute

Local

Reticulum: `./scripts/run-local.sh`

Hubs / Admin: `npm run local`

Dialog: `nvm use v18 && npm run local`

Spoke: `nvm use v16 && ./scripts/run-local-reticulum.sh`

postgREST: `./postgrest reticulum.conf` inside Reticulum folder

Open browser and access `https://localhost:4000`

dev/prod

Reticulum: `./scripts/run-dev.sh`

Hubs / Admin: `npm run dev`

Dialog: `nvm use v18 && npm run prod`

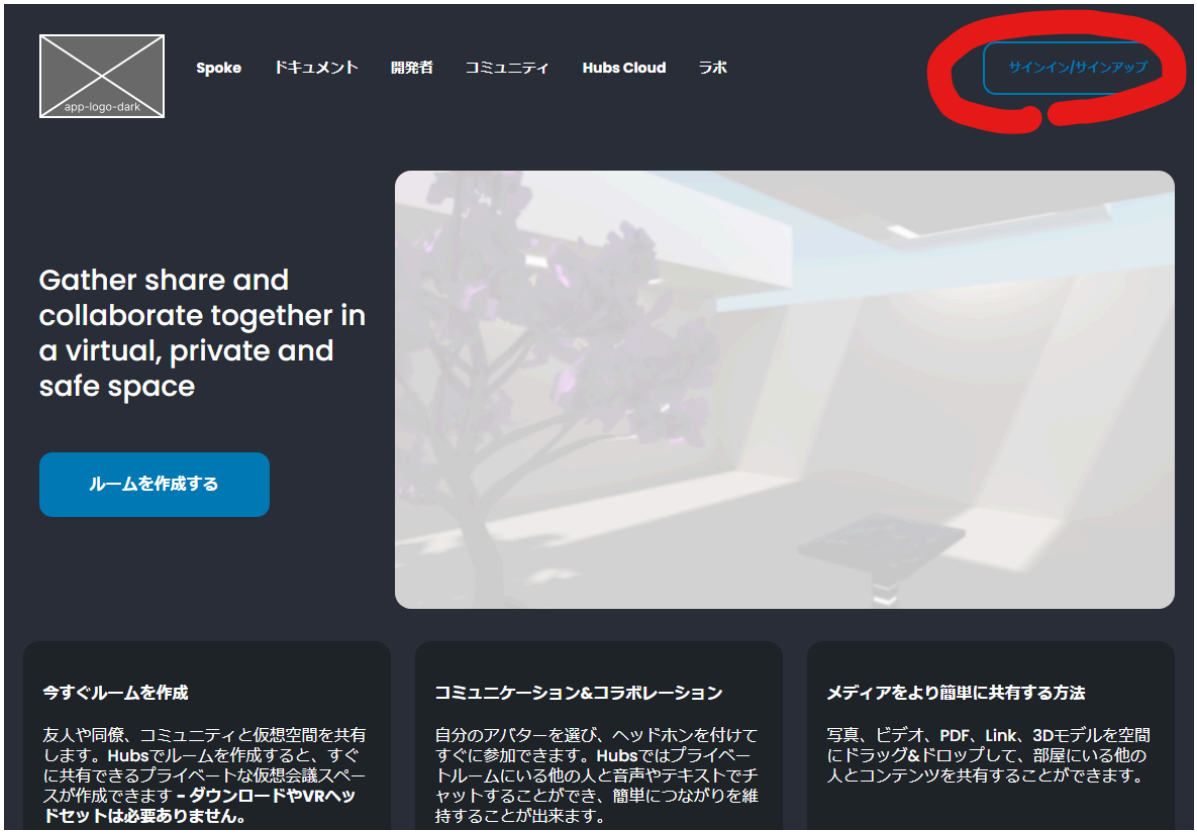
Spoke: `nvm use v16 && ./scripts/run-dev-reticulum.sh`

postgREST: `./postgrest reticulum.conf` inside Reticulum folder

Open browser and access `https://your.domain` (port number not required)

Set the first account as an admin

1. Sign up (then follow the instructions to the end)



2. Run the following lines inside reticulum iex:
   Move to the terminal where reticulum is running, press enter key, and then paste the following line, then press enter key

```
Ret.Account |> Ret.Repo.all() |> Enum.at(0) |> Ecto.Changeset.change(is_admin:
```